34. Robinhood (Stock Trading Platform)

> **Interviewer:** Let's design a stock trading app — think Robinhood. Users watch live price quotes for stocks, place buy/sell orders, and see their portfolio (cash + shares) update as orders fill. Millions of people have the app open at market open, staring at ticking prices. Design the backend.

Candidate: This is a fun one because it has *two* very different hard problems stapled together: a **massive read fan-out** (streaming prices to everyone, all day) and a **tiny, unforgiving write path** (an order touches real money and real shares, and it must never be wrong). Let me pin down scope before designing either.

---

1. Clarifying the requirements

> **Candidate:** A few questions: 1. Are we doing real order *routing* to an exchange/market-maker, or can I treat "the market" as an external system I call and get fills from? 2. Do we support only market orders, or also limit orders that can sit unfilled for a while? 3. Is a fill always atomic (fully filled in one shot), or do I need to handle **partial fills** — 30 of 100 shares now, the rest later? 4. For price streaming — do users need every single tick for every symbol, or just the symbols they're currently looking at / have on a watchlist?
>
> **Interviewer:** Treat the exchange as an external system you route orders to and get **asynchronous fill callbacks** from — you don't build the exchange itself. Support market and limit orders. Yes, partial fills are real and common. And yes — a user only needs live prices for symbols they're actively viewing or watching, not the whole market.

Candidate: Good, that scopes it nicely. Requirements:

Functional - Stream **live price quotes** for symbols a user is viewing/watchlisting. - `POST /orders` → place a buy/sell order (market or limit); idempotent on retry. - Orders move through a **lifecycle**: `PENDING → ROUTED → PARTIALLY_FILLED → FILLED` / `CANCELLED` / `REJECTED`, driven by async fill callbacks from the exchange. - Maintain each user's **portfolio**: cash balance + share positions, updated as fills land. - Users can view/edit a **watchlist** of symbols.

Non-functional - **Correctness of money and shares is non-negotiable.** We can be slow; we cannot be wrong. This needs ACID guarantees somewhere. - **Never let a user spend buying power they don't have** — including when they fire two orders at once. - **Huge read fan-out:** millions of open connections, all wanting near-real-time price updates, all day long, cheaply. - **Idempotency everywhere on the write path** — networks retry, exchanges send duplicate callbacks, users double-tap "Buy." - **Availability of the read path** (quotes) should degrade gracefully even under load; the write path (orders) must fail *safely* (reject cleanly) rather than fail *silently*.

---

2. Estimation — two very different numbers

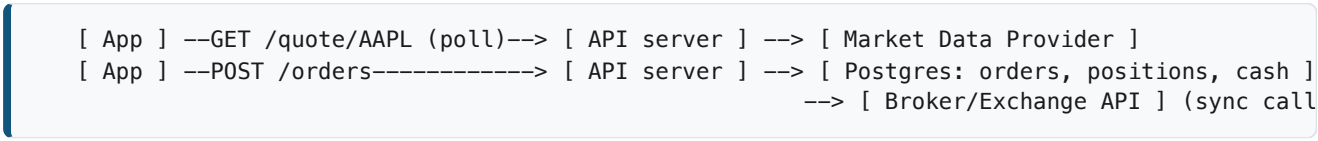
> **Candidate:** Let me size the read side and the write side separately, because they lead to completely different architectures.

```
READ SIDE (price streaming):
  10M active users, ~2M concurrently open at market open (9:30am spike).
  Each open connection is watching/viewing ~5–10 symbols on average.
  => up to ~20M "symbol interest" subscriptions, but only ~8,000 tradable
     symbols. Popular symbols (AAPL, TSLA, a meme stock) might each have
     500k–1M concurrent watchers.

  Upstream tick rate from the exchange feed: a hot symbol can tick
  10–50 times/sec during volatility. If we naively pushed every tick to
  every watcher with a per-user DB/API call:
      1 tick x 500k watchers x 20 ticks/sec = 10M outbound pushes/sec
      for ONE popular symbol alone. That's the fan-out cliff.

  => The problem is not "can we get quotes," it's "how do we fan out ONE
     tick to N subscribers without doing N units of backend work per tick."

WRITE SIDE (orders):
  2M DAU, ~5% place at least one order/day => ~100k orders/day baseline,
  but heavily clustered at open/close and around news events.
  Normal peak: a few hundred orders/sec. A "meme stock" spike (GameStop-
  style) can push this 10–50x => low thousands of orders/sec.

  Contrast with reads: orders are LOW volume but HIGH stakes — every one
  touches real money, must check buying power, must be idempotent, and
  must reconcile with an async fill from an external exchange.

=> Two designs: a fan-out/pub-sub problem for reads, and a strict-
   ledger/idempotency problem for writes. Solve them differently.
```

---

3. A simple V1

> **Candidate:** Let me sketch the smallest thing that's *correct*, then scale each side up as you raise the stakes.



- Prices: client **polls** a quote endpoint every second. Simple, correct, terrible at scale (2M clients x 1 req/sec = 2M req/sec of near-duplicate reads).
- Orders: one API call does it all synchronously — check buying power, call the exchange, wait for the fill, update the DB, return. Simple, but a slow exchange call blocks the whole request, and doing it inline holds a DB transaction open for the wrong duration.

Why it doesn't survive: polling is a read-fan-out disaster (same shape as the burger giveaway's write storm, but on reads), and a synchronous "wait for the exchange" order path means the app hangs whenever the market is choppy or the exchange is slow — exactly when users are most anxious to know their order went through.

---

4. Scaling the read path — quote streaming without linear fan-out cost

> **Interviewer:** 2M people are staring at ticking prices at 9:30am. How do you not fall over?

Candidate: Stop polling; push. And stop treating "push a tick to N people" as N units of backend work — that's the crux of read fan-out.

```
[ Exchange / market-data feed ] ── raw ticks, ~thousands/sec across all symbols
        │
        ▼
[ Ingest workers ]  normalize, then publish ONE message per tick
        │
        ▼
[ Redis Pub/Sub  (or Kafka, see below) ]  topic per symbol: "quotes.AAPL"
        │            one PUBLISH per tick, regardless of subscriber count
        ▼
[ WebSocket gateway fleet ]  (stateless, horizontally scaled, N instances)
  each instance subscribes ONLY to the symbol topics its connected
  clients currently care about, and multiplexes one tick out to every
  locally-attached socket watching that symbol
        │
        ▼
[ 2M persistent WebSocket connections ]
```

- **One publish, many deliveries.** The exchange feed writes a tick to Redis **once** per symbol per tick; Redis Pub/Sub fans it out to every subscribed gateway instance, and each gateway fans it out to its *own* locally-connected sockets. The expensive "N deliveries" work happens in cheap in-memory loops on gateway nodes, not as N separate calls into a datastore.
- **Subscribe by symbol, not by user.** A gateway instance subscribes to `quotes.{symbol}` only while at least one of its connected clients is watching that symbol — this keeps the fan-out proportional to *distinct symbols in view*, not to *users*.
- **Throttle/coalesce on the client side of the fan-out.** A hot symbol ticking 50x/sec is overkill for a human eye. The gateway coalesces to, say, 4-10 updates/sec per symbol per connection — collapsing bursts into "latest price wins" instead of forwarding every tick. This alone can cut outbound volume by 5-10x with zero perceptible UX loss.
- **Cold/idle symbols degrade to polling-on-demand.** If nobody's watching a symbol, don't keep a live subscription warm for it — pull it lazily when a client asks.

---

5. Scaling the write path — orders that touch real money

> **Interviewer:** Now the order path. Walk me through placing a buy order end to end.

Candidate: The write path is small in volume but has to be airtight. Two things must be atomic and correct: **reserving buying power** (so two concurrent orders can't spend the same dollar) and **routing exactly once** (so a retried request doesn't place two orders).

```
POST /orders   { symbol, side, qty, type, price?, Idempotency-Key: client-order-id }
       │
       ├─ 1. Idempotency check: has this client-order-id been seen before?
       │       yes -> return the ORIGINAL result, do nothing else.
       │
       ├─ 2. Atomically RESERVE buying power (see §deep-dive B):
       │       within one DB transaction: check available_buying_power >= cost,
       │       then debit a "reserved" column (not cash yet) in the SAME statement.
       │       fail -> reject INSUFFICIENT_BUYING_POWER, no reservation made.
       │
       ├─ 3. Write an `orders` row, status = PENDING_NEW, in the SAME transaction
       │       as the reservation (both commit together, or neither does).
       │
       ├─ 4. COMMIT. Return ACK to the user: "order pending" (fast).
       │
       ▼
[ Order Router / OMS ]  --submit--> [ Exchange / market-maker (FIX or REST) ]
       │                                      (async — may take ms to minutes)
       ▼
[ Fill callback / execution report ] ── FILLED / PARTIAL_FILL / REJECTED
       │
       ▼
[ Fill processor: idempotent on exchange execution-id ]
  - update order status
  - convert RESERVED buying power -> actual cash debit
  - update the position (shares) and cash ledger, atomically, in Postgres
  - publish "order-updated" event (Kafka) -> push to user via WebSocket, notifications
```

- **Reservation, not deferred check.** Buying power is decremented (as a *hold*, not a spend) in the same transaction that creates the order — never "check, then later assume it's still true." This is the same shape as the flash sale's inventory reservation, just on a ledger instead of a stock counter.
- **Idempotent order IDs.** The client generates a `client-order-id` (UUID) up front; a retry of a timed-out request reuses it. The server treats a repeat as a no-op that returns the original outcome — never a second order.
- **Fills are async and out of our control.** The exchange decides when/how a fill happens (fully, partially, over multiple messages). Our job is to process each execution report **idempotently** (keyed by the exchange's own execution ID) and update the ledger accordingly. See Deep-dive B for exactly how this can go wrong.

---

6. API + data model

Core endpoints

```
GET  /quotes/stream        (WebSocket upgrade; subscribe/unsubscribe to symbols)
GET  /watchlist                    -> list of symbols
POST /watchlist/{symbol}           -> add
DEL  /watchlist/{symbol}           -> remove
POST /orders   {symbol, side, qty, type, limit_price?, client_order_id}
GET  /orders/{id}                  -> status + fill history
GET  /portfolio                    -> cash, buying_power, positions[]
```

Data model — why relational for money, why not for ticks

```
accounts(user_id, cash_balance, reserved_buying_power, updated_at)
positions(user_id, symbol, qty, avg_cost_basis, updated_at)
orders(order_id, client_order_id UNIQUE, user_id, symbol, side, qty,
       type, limit_price, status, created_at, updated_at)
fills(fill_id, order_id, exchange_execution_id UNIQUE, qty, price, ts)
ledger_entries(entry_id, account_id, order_id, delta_cash, delta_qty,
               symbol, ts)             -- append-only, double-entry style
watchlist(user_id, symbol, added_at)
```

- **Accounts, positions, orders, fills → PostgreSQL (or a distributed SQL like CockroachDB for scale-out).** This is money and shares — it needs multi-row **ACID transactions** (debit reserved buying power AND insert the order row together), foreign-key integrity (a fill must belong to a real order), and it's comparatively low volume (thousands of writes/sec at peak, not millions). A NoSQL store would force us to hand-roll transactional guarantees we get for free here — the wrong trade for a ledger.
- **`ledger_entries` is append-only (event-sourced).** Every cash/share movement is a row, never mutated. `positions` / `accounts` are **materialized balances**, kept in sync transactionally with each ledger append. This gives us an audit trail *and* fast reads (see Deep-dive A for why we don't derive balances on the fly instead).
- **Watchlist** is a simple user→symbol mapping; a key-value or relational table both work fine — it's low-stakes, low-volume, and doesn't need ACID beyond its own row.
- **Live ticks are NOT stored in Postgres.** Market data is high-volume, append-only, time-ordered, and read almost exclusively as "recent window" or "give me candles for this range" — a **time-series store** (e.g. a TSDB, or Kafka topics + a columnar store for historical candles) fits far better than a row store tuned for point lookups and joins.

---

7. Latency, consistency, async, caching

Latency - Quote delivery: sub-second is the target; a few hundred ms of end-to-end latency (exchange → ingest → pub/sub → gateway → socket) is normal and acceptable — humans don't perceive the difference between 80ms and 250ms on a price tick. - Order ACK ("we received and reserved your order"): must be fast (a few hundred ms) because the user is waiting for confirmation. The **fill** can take longer — that's inherently async and shown as a status change, not blocked on.

Consistency - **Strong / ACID:** the ledger — cash, positions, buying-power reservations, and order status transitions. This is the one place we spend the cost of serializable transactions, because being wrong here is a compliance and money problem, not just a UX blemish. - **Eventual:** price quotes (by design — a quote is a snapshot of a moving target, and a few hundred ms of staleness is inherent to any market data feed, not a bug we're introducing) and the *display* of "portfolio value" derived from the latest

quote plus the ledger's share count (see Deep-dive A). - **Read-your-writes** for the user's own orders: after placing an order, the user must immediately see it as `PENDING`, even before any fill — we return that state directly from the transaction that created it, not from a cache that might lag.

Async — off the critical path - Fill processing, ledger updates *beyond the initial reservation*, notifications ("your order filled"), and analytics all happen behind Kafka, consuming order and execution events. The user got their `PENDING` ACK already; everything downstream of the actual fill is eventually-consistent by nature (the exchange itself is async). - Historical candle aggregation (1m/5m/1d bars) is computed by async jobs off the raw tick stream, never inline with the live path.

Caching - **Latest quote per symbol** lives in Redis (`quote:{symbol}` → last price + timestamp), written by the ingest workers on every tick, read by the WebSocket gateways and by `GET /portfolio` when computing display-only unrealized P/L. This is a cache of a fast-moving value, not a cache of a DB row — there's no "invalidation," just overwrite. - **Portfolio summary** (cash, buying power, positions) is read straight from Postgres — it's low-volume per user and must be correct, so we don't cache it; we cache the *quotes* that get combined with it at render time.

---

8. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Ledger (cash + positions + orders)	**PostgreSQL / distributed SQL (ACID)**	Multi-row transactions (reserve buying power + create order, atomically) and referential integrity are core requirements; volume is low (thousands/sec) so relational cost is affordable. *Not a NoSQL/KV store* — you'd be hand-building transactions and constraints that Postgres gives natively, on the one dataset where a bug means real money.
Market data / tick history	**Time-series DB (e.g. InfluxDB/Timescale) + Kafka as the ingest log**	Ticks are high-volume, append-only, and queried by time range/aggregation ("candles") — a TSDB is built for exactly that access pattern. *Not Postgres* — millions of tick rows/day with range-scan aggregation queries would strain a general row store; *not skipping storage* — we need historical candles for charts.
Live quote fan-out	**Redis Pub/Sub (or Kafka) + WebSocket gateway fleet**	One publish per tick reaches all interested gateways/subscribers without per-subscriber backend work; WebSockets give push instead of poll. *Not client polling* — 2M clients polling every second is a self-inflicted read storm; *not a DB read per tick* — a moving value shouldn't round-trip through a datastore tuned for durability.
Order events / fills	**Kafka**	Durable, ordered-per-order (partition by `order_id`), replayable — lets fill processing, notifications, and analytics consume independently and lets us reprocess/reconcile after an outage. *Not direct synchronous calls* from the exchange callback straight into every downstream system — a slow notification service must never block ledger updates.

Concern	Choice	Why this, and why not the alternative
Idempotency store	**Postgres unique constraint on `client_order_id` / `exchange_execution_id`**	The dedup check needs to be transactionally tied to the same commit that creates the order/fill row — a separate cache-based dedup (like the burger's `SET NX`) can't give you that atomicity with a multi-column ACID write. This is the one place we *don't* reach for Redis dedup, because the correctness boundary is the DB transaction itself.
Buying-power reservation	**A column on the `accounts` row, updated inside the same Postgres transaction as order creation**	Needs to be atomic *with* order creation (see Deep-dive B) — a separate Redis counter (great for the burger/flash-sale inventory problem) would decouple the reservation from the order row and reopen a race between "reserve" and "record".
Watchlist	**Simple relational table or KV store**	Low volume, low stakes, no transactional coupling to money — either works; pick whichever your team already operates.

The one-sentence justification you'd say out loud: "*Everything that's money or shares lives in an ACID relational ledger where reservation and order-creation commit together in one transaction; everything that's a fast-moving, re-derivable number — quotes, candles — lives in Redis/Kafka/a time-series store and is fanned out by publish-once, deliver-to-many, never by per-subscriber backend work.*"

---

9. Failure modes & graceful degradation

Failure	What happens	Mitigation
Market-data feed drops	Quotes go stale	Gateway shows "last known price, as of Ts" with a staleness indicator rather than silently freezing; auto-reconnect with backoff
Redis Pub/Sub node dies	Subscribers on that shard miss ticks briefly	Gateway resubscribes to a replica/other shard; quotes are inherently ephemeral, so a missed tick is superseded by the next one — no backfill needed
Exchange/broker API down	Can't route new orders	Reject new orders fast with a clear "trading temporarily unavailable"; never silently queue money-moving actions against an unconfirmed destination
Fill callback delayed/duplicated	Order status could go stale or double-apply	Idempotent processing keyed by `exchange_execution_id`; reconciliation job polls exchange order status for anything `ROUTED` too long (see Deep-dive B)
Kafka lag	Notifications/analytics delayed	Ledger itself is unaffected (updated synchronously in the fill-processing transaction, not via Kafka lag); user-visible order status still updates promptly
Postgres primary down	Can't place/settle orders	Fail closed: reject new orders rather than risk an unreserved buying-power spend; failover to replica;

Failure	What happens	Mitigation
		existing WebSocket quote stream is unaffected since it doesn't depend on Postgres
WebSocket gateway instance crashes	Its connections drop	Client auto-reconnects to another gateway instance via the LB; resubscribes to its watched symbols; no state lost since gateways are stateless

---

10. Follow-up curveballs

> **Interviewer:** A user has a limit order sitting unfilled for three days. Does it hold their buying power the whole time?

Candidate: Yes — that's the point of a reservation. The moment the limit order is accepted, its cost is subtracted from `available_buying_power` (held, not spent) until it fills or is cancelled. If it were only checked at placement and not held, the user could place ten unfillable limit orders that collectively exceed their cash, then have several fill simultaneously when the market moves — an oversell of buying power exactly like overselling inventory. Cancelling the order releases the hold atomically, same shape as the flash sale's TTL release.

> **Interviewer:** How do you show "portfolio value" updating live as prices move, without hammering the database?

Candidate: Portfolio value = `cash + Σ(shares_held × latest_price)`. The share counts and cash come from the ledger (read once, changes rarely); the *prices* come from the same Redis quote cache the streaming path already maintains. We recompute this client-side or in a thin aggregation layer on each quote tick for symbols the user holds — never a fresh DB query per tick. This is a "cheap render," not a ledger read.

> **Interviewer:** What about market open, when literally everyone reconnects at 9:30am sharp?

Candidate: Same reconnection-storm shape as any "everyone shows up at once" system — jittered client reconnect (small random delay), gateway fleet auto-scaled ahead of the known market-open time, and admission control at the load balancer so a burst of reconnects degrades to "brief delay in quote stream" rather than cascading gateway failures. This doesn't touch the ledger at all, so it's a pure read-path scaling problem, not a correctness one.

> **Interviewer:** Regulators want an audit trail of every order and every price the user saw when they placed it. Do you have that?

Candidate: Orders and fills are already durable rows; I'd add a lightweight snapshot at order-placement time — the last quote price the client displayed, stored alongside the order row (denormalized, cheap) —

specifically for audit/dispute purposes. It's not used for any correctness logic (the ledger doesn't depend on it), purely a compliance record.

---

11. Deep-dive: "Why not compute portfolio value from trade history on demand, and why not

push every tick synchronously?"

> **Interviewer:** Why maintain a materialized `positions` / `cash_balance` at all? Couldn't you just replay a user's entire trade history on each `GET /portfolio` and derive the current state? And on the quotes side — why not just push every tick directly to every connected client the instant it arrives, synchronously?

Candidate: Both are the same mistake in different clothing: **doing O(n) work on every read, where n grows with either history or subscriber count, when you could do the work once and let reads be O(1).**

Why not derive the portfolio from trade history on every request

```
Naive: GET /portfolio ->
  SELECT * FROM ledger_entries WHERE user_id = ? ORDER BY ts
  fold over every entry to compute current cash + shares per symbol

A user active for 3 years, trading a few times a week, might have
5,000-50,000 ledger entries. Replaying that on EVERY portfolio view
(which users check constantly - it's the app's home screen) means:
  - a full table scan / large index range-scan per request
  - CPU spent re-deriving a value that changed by exactly one entry
    since the last time anyone looked
  - this is the single most-viewed screen in the app, so it's also the
    highest-QPS query -> the worst possible place to pay O(n) cost
```

The fix is exactly what accounting systems have always done: **the ledger is the source of truth, but you maintain a materialized running balance** (`accounts.cash_balance`, `positions.qty`) updated transactionally alongside every ledger append. Reads become O(1) row lookups; the append-only ledger still exists underneath for audit/replay/dispute resolution — you get correctness *and* cheap reads, not one or the other.

Why not push every tick synchronously to every client

```
Naive: on each tick, iterate the list of subscribers and call
       websocket.send() for each one, inline, in the ingest path.

A hot symbol with 500k watchers, ticking 20x/sec:
  500k sends x 20/sec = 10M synchronous send calls/sec attributable
  to ONE symbol, all originating from whatever process received the
  tick. That process becomes the bottleneck for the entire symbol,
  and a slow/stuck subscriber's send() call can stall the whole loop
  for everyone else behind it.
```

Publish-once/deliver-to-many (pub/sub) turns this into: **one publish**, fanned out by the messaging layer to however many gateway processes are subscribed, each of which does its *own* local, independent fan-out to its *own* connections — parallelized and isolated, so one slow client can't stall another's delivery.

Concern	Compute-on-read / push-per-tick-synchronously	Materialized ledger + pub/sub fan-out
Cost per read/tick	Grows with history length or subscriber count — O(n)	O(1) read of a maintained balance; O(1) publish regardless of subscriber count
Blast radius of a slow consumer	One slow websocket.send() can stall the whole tick-processing loop	Isolated per gateway process; a slow client only affects itself
Where correctness lives	Recomputed fresh every time — technically "obviously correct" but expensive	Materialized value kept correct via the *same* transaction that appends the ledger entry — correctness is enforced once, at write time
Staleness risk	None (always fresh) — but you pay for that freshness every single read	None either — the materialized balance is updated in the same transaction as the write that changed it, so it's never behind
Scales with	Number of reads x history size / number of subscribers	Number of *writes* (rare) for the ledger; number of *distinct symbols* for fan-out, not number of subscribers

> **Rule of thumb:** *If a value is read far more often than it changes, maintain it — don't derive it. If one event needs to reach many listeners, publish once and let a fan-out layer multiply the delivery — don't let the producer pay for every consumer synchronously.*

---

12. Deep-dive: order execution correctness — delayed/duplicate fills and buying-power

double-spend

> **Interviewer:** Walk me through the scariest failure mode: a user submits an order, it's routed to the exchange, and the fill confirmation is **delayed and then arrives twice**. Separately — what stops a user from placing **two orders concurrently** that both draw on the same buying power?

Candidate: These are two instances of the same root issue — **an operation that isn't naturally atomic across a network boundary needs an explicit mechanism to become atomic in effect.** Let me take them one at a time, then show how they interact.

Timeline 1 — the buying-power double-spend

```
Account has $1,000 buying power. User has two devices/tabs open (or a
flaky app that retries).

t=0.000  Request A: BUY 100 AAPL @ ~$10  (cost ~$1,000) arrives at server 1
t=0.001  Request B: BUY 100 TSLA @ ~$10  (cost ~$1,000) arrives at server 2
         Both check "available buying power >= cost" against the SAME
         $1,000 balance, in separate connections, near-simultaneously.

WITHOUT atomic reservation:
  server 1: reads buying_power = $1000 -> $1000 >= $1000 -> OK, proceeds
  server 2: reads buying_power = $1000 -> $1000 >= $1000 -> OK, proceeds
  both write orders, both later route, both could fill.
  => user has spent $2,000 of buying power they only had $1,000 of.
     This is a classic check-then-act race — read-modify-write done as
     two separate steps instead of one atomic step.
```

Timeline 2 — the delayed/duplicate fill

```
t=0:00.0  Order X (100 AAPL) reserved + routed to the exchange.
          orders.status = ROUTED

t=0:00.5  Exchange fills it. Sends an execution report... but the
          response is slow over a flaky link.

t=0:04.0  Our reconciliation job, seeing ROUTED for 4 seconds with no
          fill, gets nervous and POLLS the exchange for order status
          (a reasonable safety net) -- gets back "FILLED, 100 shares."
          It applies the fill: position += 100, cash -= cost.
          orders.status = FILLED

t=0:04.2  The ORIGINAL execution report (delayed, not lost) finally
          arrives from the exchange -- same fill, same execution ID,
          now processed a SECOND time by a naive handler.
          => position += 100 AGAIN. User appears to own 200 shares
             they only bought 100 of. Classic double-apply from a
             duplicate delivery.
```

The bug in both timelines is the same shape: **treating a check (or a "have I seen this already?") and the action that follows it as two separate, non-atomic steps**, leaving a window where a second actor — a concurrent request, or a redelivered message — sneaks through.

Ranked fixes

1. **Reserve buying power atomically, in the same transaction that creates the order (mandatory).** `UPDATE accounts SET available = available - cost WHERE user_id = ? AND available >= cost` — a single conditional statement, not a separate read followed by a separate write. If the `UPDATE` affects 0 rows, the check failed and nothing was reserved; if it affects 1 row, the reservation and the check happened as one atomic database operation. Two concurrent requests hitting this on the same account are serialized by the database's own row-level locking — exactly the same principle as the flash sale's atomic `DECR`, just expressed as a conditional `UPDATE` inside an ACID transaction because this value (buying power) *must* live in the ledger, not in a separate fast cache.

2. **Idempotent order IDs and idempotent fill processing (mandatory).** Every order carries a client-generated `client_order_id` with a uniqueness constraint — a retried "place order" request is a no-op, not a second order. Every fill is processed keyed by the exchange's own `execution_id`, also uniquely constrained — a redelivered execution report is detected and dropped *before* it touches the ledger, not after. This directly closes Timeline 2: the second delivery of the same execution report hits a unique-constraint violation (or an explicit "already processed" check) and is discarded.
3. **Async fill reconciliation as a safety net, not the primary mechanism.** A background job polls the exchange for any order stuck in `ROUTED` / `PARTIALLY_FILLED` past an expected window, to catch fills whose callback never arrives (as opposed to arriving late/twice). Because fill processing is idempotent (fix #2), it's safe for reconciliation and the original callback to race — whichever arrives, the other is a harmless no-op.
4. **Treat partial fills as their own idempotent events, not a mutation of "the" fill.** A single order can generate multiple fill events (30 shares, then 70 shares). Each is a distinct `execution_id` and a distinct ledger append — don't model "the order's fill" as a single mutable field that a duplicate message could stomp; model it as an append-only set of fill events that sum up to the order's filled quantity.

How the two failures interact

Notice fix #1 also protects against a subtler version of Timeline 2: if a duplicate fill somehow *did* get past the idempotency check, the damage would be a ledger error (positions overstated), not a buying-power breach — because buying power was already converted from "reserved" to "spent" at the *first, legitimate* fill. Layering both fixes means a failure in one layer doesn't compound into the other.

Recommended approach

Lead with **#1 as the non-negotiable invariant for buying power** — no order is created without an atomic, same-transaction reservation, full stop, exactly the same principle as never charging a payment without a freshly-validated hold. Pair it with **#2 as the non-negotiable invariant for order/fill identity** — every order and every fill has a unique key that makes retries and redeliveries safe by construction. Layer **#3** underneath as the net that catches fills whose *first* delivery never arrives, and use **#4** to keep partial fills correct under all of the above.

> **What to say out loud:** "*Buying power is reserved with one atomic conditional update in the same transaction as order creation, so two concurrent orders can never both pass a stale check — the database's own row lock serializes them. Every order and every fill carries a unique idempotency key, so a retried request or a redelivered execution report is a detected no-op, never a double-apply. A reconciliation job is the safety net for fills that never arrive at all, not the primary mechanism — and because everything downstream is idempotent, it's safe for the original callback and the reconciliation poll to race.*"

Summary — the mental model

Robinhood is really **two systems wearing one UI**: a **read fan-out problem** — stream a small number of fast-moving prices to millions of watchers by publishing once and letting a gateway fleet multiply delivery, never by doing per-subscriber work in the hot path — and a **strict-ledger problem** — every dollar and every share moves through ACID transactions where buying power is reserved atomically in the same statement that creates an order, every order and fill carries an idempotency key, and async fills from the exchange are reconciled against a durable, append-only ledger rather than trusted blindly. Materialize what's read often and changes rarely (portfolio balances); publish once and fan out cheaply what's read by many and changes constantly (quotes). Keep money exactly where atomicity is native — a relational ledger — and keep everything else eventually consistent by design, not by accident.